Robust Training of Neural Networks at Arbitrary Precision and Sparsity

C. Ye, G. Chu, Y. Liu, Y. Zhang, L. Lew, L. Zhang, M. Sandler, A. Howard
Google DeepMind • ICLR 2026

Presented by

Ayush Srivastava (2026AIB2015) Sarvesh Nikas (2026AIB2045)

Why quantization, and the forward pass during training

Why quantization. Modern neural networks are huge: a language model has billions of weights, each stored as a 32-bit or 16-bit floating-point number. This makes models too big to fit on a phone, slower to run, and costly in energy. Quantization stores each weight with far fewer bits: 4, 2 or 1 bit instead of 16 or 32.

Training with quantization. During training the network keeps the float weights x and updates them by gradient descent. We could compute with x, but we deliberately use the quantized version $\hat{x}$ in the forward pass, so the network learns to be robust to the rounding it will face at inference. After training, x is discarded; only the low-bit integers q (plus one s_f, b_f per block) are kept.

The forward pass during training.

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{s_f q + b_f \mathbf{1}} \hat{x} \longrightarrow \text{rest of network} \longrightarrow L$$

- $x \in \mathbb{R}^N$: float block, kept and updated during training.
- $f(x) = \dfrac{x - b_f \mathbf{1}}{s_f}$: rescales x onto the k-bit integer grid $\{0, \ldots, 2^k - 1\}$, with $s_f = \dfrac{x_{\max} - x_{\min}}{2^k - 1}$, $b_f = x_{\min}$.
- $q = \text{round}\big(f(x)\big) = f(x) + \delta$: the integers, with rounding error $|\delta_i| \leq \frac{1}{2}$.
- $\hat{x} = s_f\, q + b_f \mathbf{1} = x + s_f\, \delta$: x as seen after quantization; this is what the next layer uses.

The problem: the true gradient is zero

Chain rule through the quantizer. Gradient descent needs $\nabla_x L$:

$$\nabla_x L = \Big(\frac{\partial \hat{x}}{\partial x}\Big)^{\top} \nabla_{\hat{x}} L, \qquad \frac{\partial \hat{x}}{\partial x} = \underbrace{s_f}_{\text{scale back}} \cdot \underbrace{\frac{\partial q}{\partial f(x)}}_{\text{round}} \cdot \underbrace{\frac{1}{s_f}}_{f} = \frac{\partial q}{\partial f(x)}.$$

The two scalings cancel: the local gradient is exactly the slope of round.

Round is a staircase.

$$\frac{d}{du}\,\mathrm{round}(u) = 0 \quad \text{for } u \notin \mathbb{Z} + \tfrac{1}{2}, \qquad \text{undefined at } u \in \mathbb{Z} + \tfrac{1}{2}.$$

Hence $\dfrac{\partial q}{\partial f(x)} = 0$ almost everywhere, so

$$\nabla_x L = 0.$$

Consequence. A small change in x does not change q, so the loss gives x no direction to move: x and every layer before it never update. Training cannot start.

The standard fix: Straight-Through Estimator (STE)

Idea. Round as before in the forward pass, but treat the rounding error as a constant in the backward pass:

$$q = f(x) + \delta, \qquad \delta = \text{stopgrad}\Big(\text{round}\big(f(x)\big) - f(x)\Big),$$

where stopgrad keeps the value in the forward pass and has zero derivative in the backward pass.

Forward vs. backward.

$$\text{forward:}\quad \hat{x} = x + s_f\,\delta, \qquad\qquad \text{backward:}\quad \frac{\partial \hat{x}}{\partial x} = s_f \cdot I \cdot \frac{1}{s_f} = I, \quad \nabla_x L = \nabla_{\hat{x}} L.$$

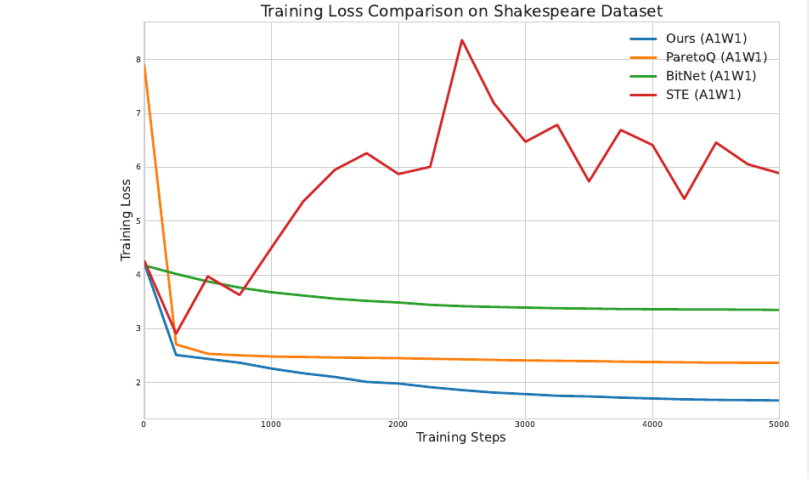
The gradient reaching x is no longer zero, so training works.

The blind spot.

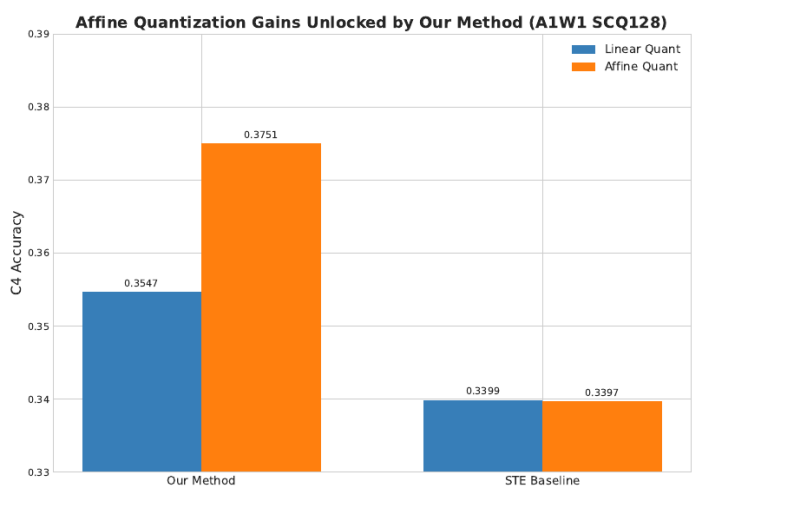
- The forward pass shifts every entry by $s_f\delta_i$, but the Jacobian is I, exactly as with no quantization. δ does not appear in it.
- So earlier layers get no signal about how rounding hurts the loss, and cannot learn to be robust to it.
- At 8 bits $s_f\delta$ is tiny and this is harmless. At 1–2 bits it is as large as the values themselves: STE diverges, and gains nothing from affine quantization (next slide).

Reference: Y. Bengio, N. Léonard, A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," arXiv:1308.3432, 2013.

STE at 1 bit: results from the paper



Training loss, 1-bit weights and activations (A1W1), small GPT on Shakespeare. STE (red) diverges; the paper's method (blue) converges.



Accuracy at A1W1, linear vs. affine quantization. Under STE affine gives no gain; with the paper's method it does.

Source: Ye et al., "Robust Training of Neural Networks at Arbitrary Precision and Sparsity," ICLR 2026, Fig. 1.

The proposed approach: a denoising reconstruction g

The change. Keep f and round. Replace $\hat{x}$ by a fitted reconstruction g.

Reconstruction.
A new affine map.

$$\hat{x} = s_f\, q + b_f \mathbf{1} \quad \downarrow \quad g(q) = s_g\, q + b_g \mathbf{1}$$

$\Longrightarrow$

Ridge regression objective.
Fit s_g and b_g to x.

$$\min_{s_g,\, b_g} \frac{1}{2N} \left\| s_g\, q + b_g \mathbf{1} - x \right\|^2 + \frac{\lambda}{2}\, s_g^2$$

$\Longrightarrow$

Solution.
Ridge regression has a closed form.

$$s_g = \frac{\mathrm{Cov}_{xq}}{\mathrm{Var}_q + \lambda}, \qquad b_g = \bar{x} - s_g\, \bar{q}$$

λ *is the denoising knob.*

Training with g. One forward and one backward pass.

Forward pass. The next layer gets $g(q)$, not $\hat{x}$.

$$x \xrightarrow{f} f(x) \xrightarrow{\text{round}} q \xrightarrow{g} g(q) \longrightarrow \text{rest of network} \longrightarrow L$$

Backward pass. Round stays straight through, as in STE. But g now depends on δ.

$$\nabla_x L \xleftarrow[\text{straight through}]{\times\, 1/s_f} \nabla_q L \xleftarrow[\text{depends on } \delta]{\times\, (\partial g/\partial q)^\top} \nabla_g L \longleftarrow \text{rest of network} \longleftarrow L$$

Why it works: the backward pass sees δ

The idea. Both methods replace round's slope by a constant surrogate, so $\partial q/\partial x = I/s_f$. Only the step after round differs.

STE. $\hat{x} = s_f\, q + b_f \mathbf{1}$ has the same slope for every δ.

$$\frac{\partial \hat{x}}{\partial q} = s_f\, I \quad \Longrightarrow \quad \nabla_x L = \nabla_{\hat{x}} L$$

With g. The gradient reaches x by two paths.

$$\nabla_x L = \underbrace{\frac{1}{s_f}\Big(\frac{\partial g}{\partial q}\Big)^\top \nabla_g L}_{\text{through } q} + \underbrace{\Big(\frac{\partial g}{\partial x}\Big)^\top \nabla_g L}_{\text{through } x}$$

- **Through q.** Changes with the rounding error. It carries δ into the gradient:

$$\frac{\partial g}{\partial q} = \underbrace{s_g\, I}_{\substack{\text{like STE's } s_f I, \\ \text{but } s_g \text{ carries } \delta}} + \underbrace{q\Big(\frac{\partial s_g}{\partial q}\Big)^\top + \mathbf{1}\Big(\frac{\partial b_g}{\partial q}\Big)^\top}_{\substack{\text{new terms,} \\ \text{carry } \delta \text{ through } x - g(q)}}$$

- **Through x.** For the current q, g is linear in x. *No information about the rounding error.*

Results from the paper

Gemma 3 language model (1B), trained from scratch. Score: share of next tokens predicted correctly.

bits for weights and activations	STE	with g	gain
4	0.425	0.427	+0.002
2	0.379	0.403	+0.023
1	0.340	0.375	+0.035

The fewer the bits, the bigger the gain.

- **STE can diverge.** At 1.5 bits $(-1, 0, 1)$, one STE run ends in NaN. g still trains, reaching 0.330.

Bigger model, fewer bits.
Quantized Gemma 3 4B beats 1B.

- 4B, 1-bit weights, half pruned: 650 MB, score 0.452.
- 1B, full precision: 1400 MB, score 0.449.

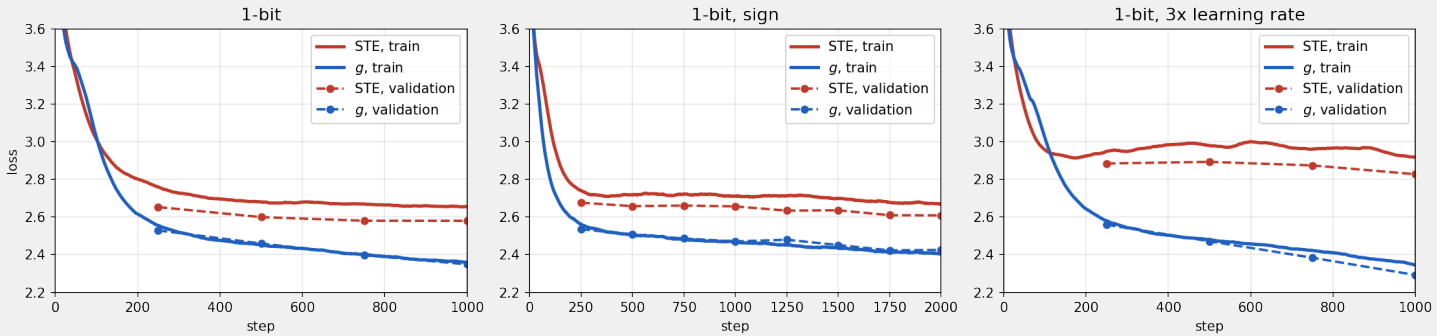
Other tasks, 4 bits, no extra tuning.
Matches full precision.

- Images (ResNet-50): 76.45% vs. 76.41%.
- Translation (English–German): 29.71 vs. 29.49 BLEU.

Our implementation: training at 1 bit

Setup. Small GPT on Shakespeare, 1-bit weights and activations.

Loss at 1 bit: STE stalls, g keeps learning



- **STE stalls, g keeps learning.** In all three runs.
- **STE's activations blow up.** About 340× too large.

- **At 3× the learning rate.** STE's training loss rises.
- **Reason.** Bigger steps on a rounding-blind gradient.

Reference: model and recipe follow nanoGPT by A. Karpathy, `github.com/karpathy/nanoGPT`.

Advantages and disadvantages

Advantages.

- **Makes 1-bit training work.**

  1-bit matmul is just XNOR + bit-count,
  256× *cheaper than 16-bit.*

- **Better reconstruction.**

  Closer to x than standard dequantization.

- **Extends to sparsity.**

  Quantization with small values set to 0:
  $x \xrightarrow{\text{threshold}} x_S \xrightarrow{f,\ \text{round}} q \xrightarrow{g} g(q)$

Disadvantages.

- **Round still uses a surrogate.**

  "Without gradient estimation" is too strong.

- **Small gain at 4 bits.**

  Only +0.002, *in the most common setting.*

- **Savings are estimates.**

  No real hardware, no speed numbers.

Our takeaways

What convinced us.

- **A clean comparison.**

  Only the reconstruction differs between the two methods.

- **Consistent gains.**

  g wins in every setting, more at fewer bits.

- **Our 1-bit run agrees.**

  STE stalls; g keeps learning.

What we would question.

"No gradient estimation".

The paper claims it (abstract; Table 8).

But δ is detached, exactly as in STE:

$$q = f(x) + \text{stopgrad}(\delta) \;\Rightarrow\; \partial q / \partial f(x) = I$$

Takeaway

A good approximation can still cause optimization issues.

Rounding error looks ignorable, but training needs its information.

Fit parameters to the data wherever possible.